
EduGenie – Google Gemini Powered Learning Assistant

Phase 3 – Project Design

System Architecture, Module Design, Workflow and Interface Design

Table of Contents

1. Introduction	3
2. System Design Overview	3
3. System Architecture	4
4. System Workflow	5
5. Module Design	5
6. Frontend Design	6
7. Backend Design	7
8. AI Model Design	7
9. API Design	8
10. Data Flow Design	8
11. Security Design	9
12. System Design Summary	9
13. Conclusion	9

1. Introduction

The Project Design phase describes the structure of the EduGenie system and the way its parts interact. It builds on the Requirement Analysis phase and translates the identified requirements into a technical structure. In EduGenie, this structure consists of the following elements:

- Frontend
- Backend
- Learning modules
- AI model integration
- API endpoints
- User interaction flow

This document is based on the current EduGenie implementation. Where a capability is not part of the current project, it is stated as not part of the current design and no design is proposed for it.

2. System Design Overview

EduGenie follows a simple web-based architecture. A request from the student passes through the frontend to the backend, is handled by the appropriate learning module and AI component, and the generated response is returned to the interface.

Overall flow: User → Frontend → FastAPI Backend → Appropriate Learning Module → AI Model / API → Response → Frontend

Table 1: Layers of the EduGenie design

Layer	Description
User Layer	The student provides a question, topic, concept, or learning content.
Frontend Layer	The HTML/CSS/JavaScript interface allows the user to select a task, enter input, submit the request, and view the generated result.
Backend Layer	FastAPI receives the request and routes it to the appropriate functionality.
Learning Module Layer	The request is handled by the corresponding module: Q&A, Explanation, Quiz, Summary, or Learning Path.
AI Layer	Depending on the task, the system uses Gemini 1.5 Pro through an API or the local LaMini-Flan-T5-783M model.
Response Layer	The generated result is returned to the frontend and displayed to the user.

Scope of the current design. Based on the project information used for this document, the following are not part of the current design: a database or persistent data storage, user registration and authentication, payment features, analytics, and dedicated deployment or cloud infrastructure. Features described only as future enhancements are likewise not presented as part of the system.

3. System Architecture

Figure 1 presents the architecture of EduGenie. The student interacts only with the web interface, which sends requests to the FastAPI backend. The backend directs each request to one of five learning modules. The Explanation module uses the local LaMini-Flan-T5-783M model, while the Q&A, Quiz, Summary, and Learning Path modules use the Gemini 1.5 Pro API. The generated learning response is returned to the web interface and shown to the student.

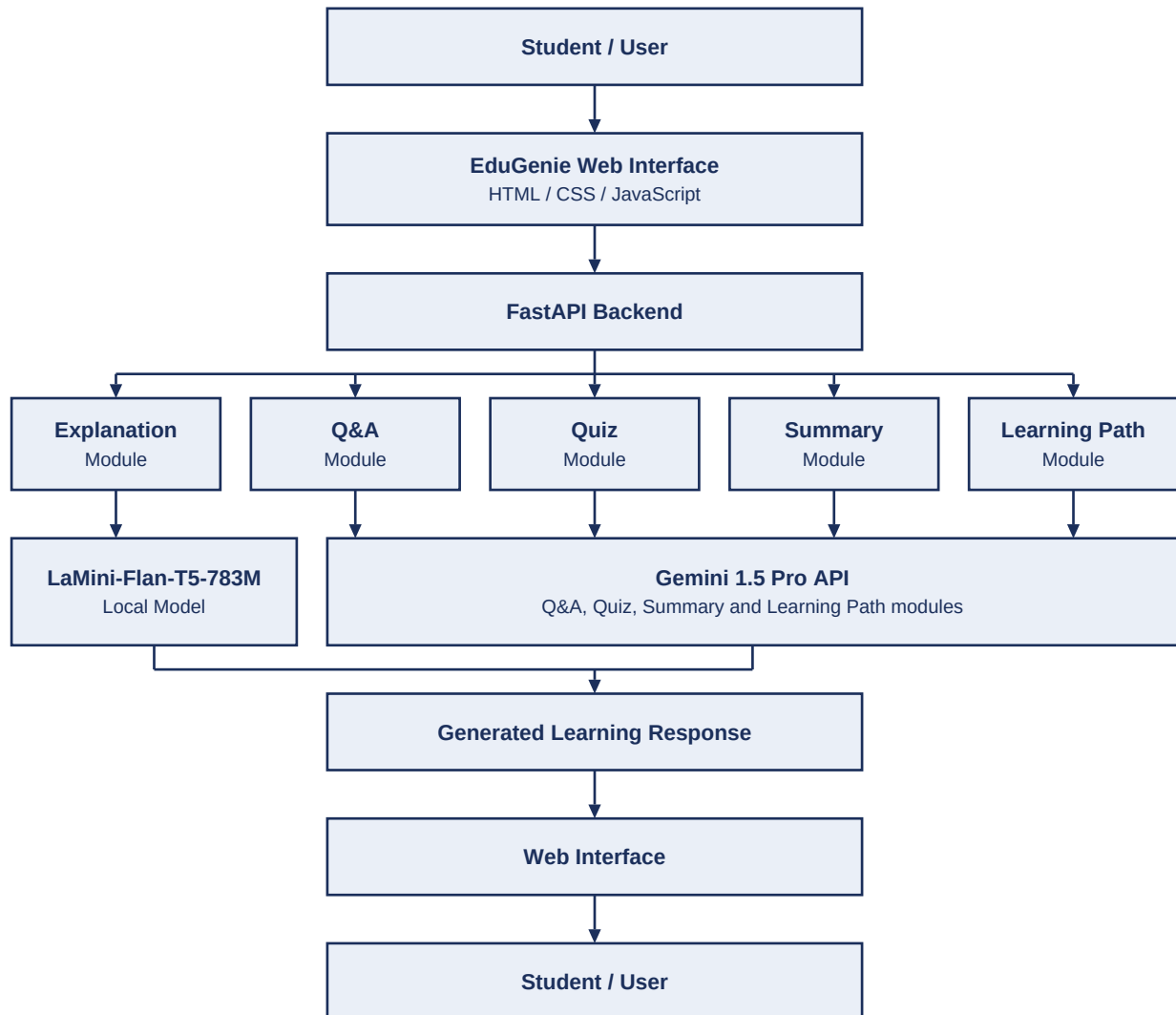


Figure 1: EduGenie System Architecture

The architecture contains no database or other persistent storage component.

4. System Workflow

Figure 2 shows the normal operation of the application, from opening EduGenie to viewing the generated result.

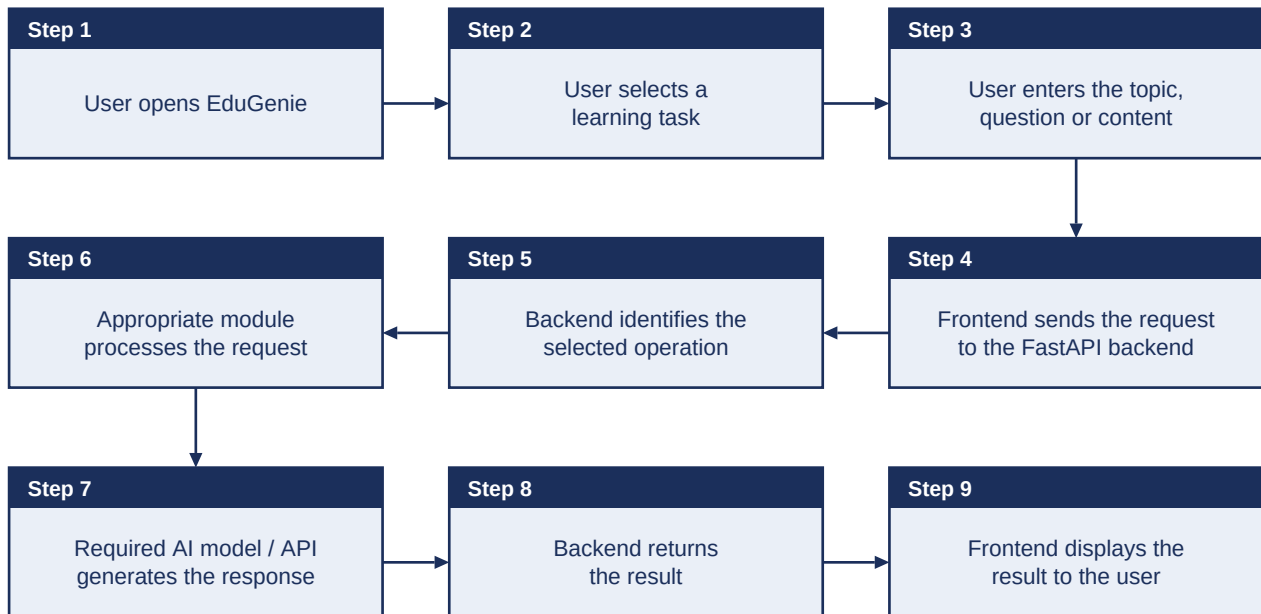


Figure 2: EduGenie System Workflow

The user opens EduGenie, selects a learning task, and enters the required topic, question, or content. The frontend sends the request to the FastAPI backend, which identifies the selected operation and passes the input to the corresponding module. The module obtains the response from the required AI model or API. The backend returns the result to the frontend, which displays it to the user.

5. Module Design

The project uses modular Python files for the different learning functions. Each function is described below together with its endpoint and basic flow.

5.1 Q&A Module

The Q&A module accepts a user's educational question and generates a concise answer using the configured Gemini-based functionality.

Associated endpoint: /qa

Basic flow: Question → Backend → Q&A Processing → Gemini → Answer → User

5.2 Explanation Module

The Explanation module provides simplified explanations of concepts. The current project uses the local LaMini-Flan-T5-783M model for concept explanation.

Associated endpoint: /explain

Basic flow: Concept → Backend → Explanation Module → Local Model → Simplified Explanation → User

5.3 Quiz Module

The Quiz module generates quiz questions for learning practice. The current implementation generates:

- Three questions
- Four options for each question
- Answer checking

Scoring systems, progress tracking, databases, and adaptive quizzes are not part of the current design.

Associated endpoint: /quiz

Basic flow: Topic → Backend → Quiz Module → Gemini → Quiz → User

5.4 Summary Module

The Summary module processes user-provided content and generates a concise summary.

Associated endpoint: /summarize

Basic flow: Content → Backend → Summary Module → Gemini → Summary → User

5.5 Learning Path Module

The Learning Path module provides resources and stepwise guidance for learning a selected topic. The recommendations are generated for the topic entered by the user; personalization based on stored user history is not part of the current design.

Associated endpoint: /learn/recommendations

Basic flow: Topic → Backend → Learning Path Module → Gemini → Recommendations → User

6. Frontend Design

The frontend is built using HTML, CSS, and JavaScript. It provides a single interface through which the user selects a learning function, supplies input, and views the result. Table 2 lists its main interface elements.

Table 2: Main elements of the EduGenie frontend

Element	Description
Task Selection	The user selects one of the available learning functions: Explain, QnA, Quiz, Summary, or Recommend Path.
User Input	The user enters the relevant question, topic, concept, or content.
Submit Action	The request is sent to the backend.
Result Area	The generated response is displayed on the interface.

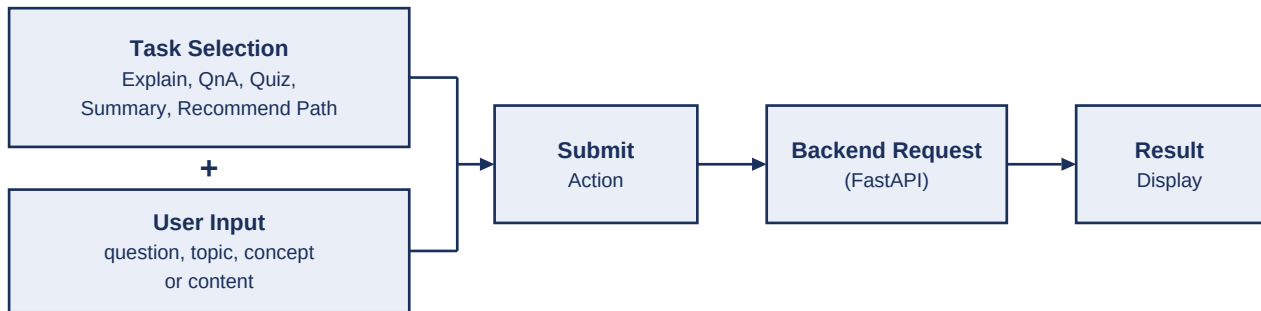


Figure 3: EduGenie Frontend Interaction Flow

7. Backend Design

The backend is built using **FastAPI**. Its responsibilities are to:

- Receive frontend requests
- Identify the requested operation
- Pass the input to the appropriate module
- Interact with the required AI model or API
- Return the generated result

The backend exposes the following endpoints, one for each learning function.

Table 3: EduGenie backend endpoints

Endpoint	Purpose
/qa	Question answering
/explain	Concept explanation
/quiz	Quiz generation
/summarize	Content summarization
/learn/recommendations	Learning recommendations

The HTTP methods and the request and response schemas of the endpoints are not specified in this document.

8. AI Model Design

EduGenie uses two AI components, each assigned to specific functions.

Gemini 1.5 Pro

Gemini 1.5 Pro is used through an API for the following functions:

- Question answering
- Summarization
- Quiz generation
- Learning recommendations

The API key is supplied through environment configuration and should not be exposed in the source code.

LaMini-Flan-T5-783M

The LaMini-Flan-T5-783M model runs locally and is used for concept explanation. Using a local model for this function allows the explanation component to operate using the locally configured model. The local model is not used for the other learning functions.

9. API Design

The frontend communicates with the FastAPI backend through the application endpoints listed in Table 4. Each learning function is exposed through its own endpoint, so the task selected in the interface corresponds to one of these endpoints. Detailed JSON request and response formats are not defined in this document.

Table 4: EduGenie API design

Endpoint	Function	Main Purpose
/qa	Q&A	Provides answers to educational questions
/explain	Explanation	Simplifies concepts
/quiz	Quiz	Generates practice questions
/summarize	Summary	Summarizes learning content
/learn/recommendations	Learning Path	Provides learning recommendations and guidance

10. Data Flow Design

Figure 4 shows how data moves through EduGenie. The user input is processed according to the selected learning function, and the resulting output is returned to the user interface. The current application does not implement persistent data storage, so no storage stage appears in the flow.

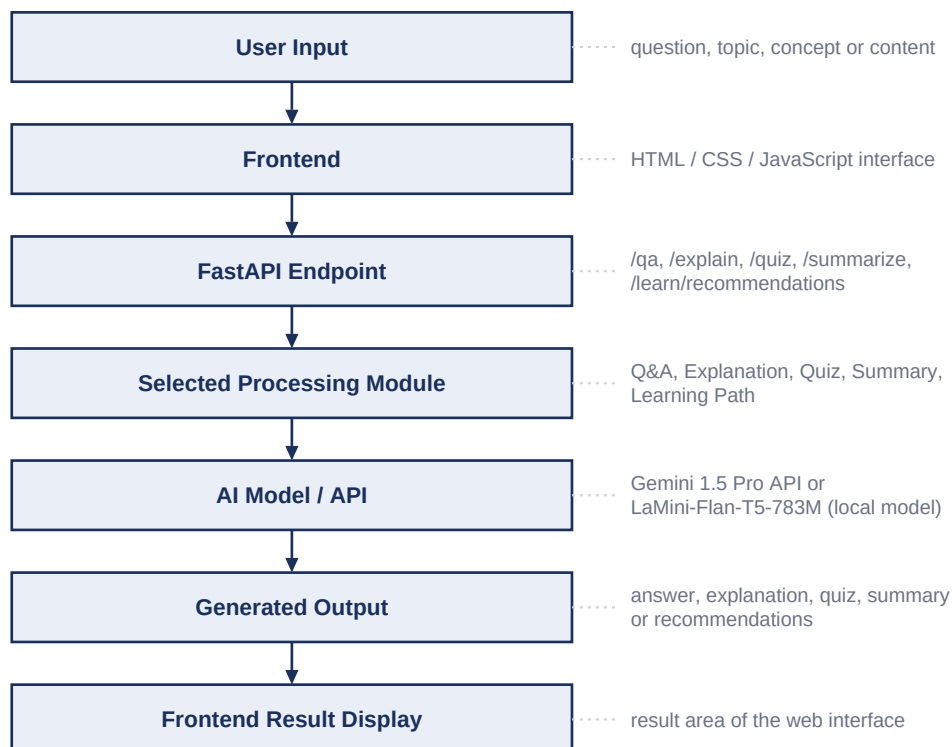


Figure 4: EduGenie Data Flow

11. Security Design

This section covers the security considerations relevant to the current project.

API Key Protection

The Gemini API key should not be hard-coded into the project or committed to GitHub.

Environment Variables

Sensitive credentials should be stored using environment variables.

.env.example

The project provides an example environment configuration `.env.example` that shows the required settings without exposing the actual secret key.

GitHub Security

Actual API keys and other secrets should not be uploaded to the public repository.

Authentication, authorization, encryption mechanisms, and security certifications are not part of the current design.

12. System Design Summary

Table 5 summarizes the technologies and design choices of EduGenie.

Table 5: Summary of the EduGenie system design

Component	Technology / Design
Frontend	HTML, CSS, JavaScript
Backend	FastAPI
Q&A	Gemini 1.5 Pro API
Explanation	LaMini-Flan-T5-783M
Quiz	Gemini 1.5 Pro API
Summary	Gemini 1.5 Pro API
Learning Recommendations	Gemini 1.5 Pro API
Communication	Frontend to FastAPI endpoints

13. Conclusion

The Project Design phase establishes the technical structure of EduGenie. The design connects the user interface, the FastAPI backend, the learning modules, and the AI components into a single educational workflow.

This design provides the foundation for the implementation and development phase.